THORN: Transport-Aligned Heterogeneous Observer-Routed Neighborhood Attention

Richard Liu
University of North Carolina at Charlotte
wliu23@charlotte.edu

February 2026

Abstract

Graph attention networks route information using content-based query-key similarity, ignoring the geometric structure of the graph itself. We propose THORN (**T**ransport-aligned **H**eterogeneous **O**bserver-**R**outed **N**eighborhood Attention), which routes attention heads to competing graph views—feature-based, temporal, and spectral—using geometric descriptors of local structure (curvature, intrinsic dimension, density) rather than learned content features. A differentiable log-barrier mask continuously relaxes hard neighborhood constraints, and a formal analysis reveals a previously uncharacterized *isolation-versus-interaction trade-off*: mixing views before versus after the attention softmax yields complementary strengths in different data regimes. On a controlled temporal-graph benchmark, post-softmax mixing achieves 0.984 PR-AUC / 0.992 ROC-AUC (+6.5 points over GAT) when one view dominates, while pre-softmax mixing achieves 0.939 PR-AUC (+2.0 points over GAT) when views are complementary. Ablation studies confirm that geometric observers—especially curvature—carry routing signal that content features cannot replace: removing curvature alone degrades performance by 7.5%. THORN subsumes GAT, Graphormer, and MoE as special cases under parameter-setting reductions.

1 Introduction

Graph neural networks (GNNs) have become the dominant paradigm for learning on relational data, with graph attention networks [Veličković et al., 2018, Brody et al., 2022] extending the transformer attention mechanism [Vaswani et al., 2017] to irregular graph topologies. A key limitation of existing graph attention methods is their reliance on *content-based routing*: attention weights are computed solely from learned query-key projections of node features, with graph structure entering only as a binary mask that restricts the attention neighborhood.

This design has three fundamental shortcomings. First, **hard neighborhood masks are non-differentiable** with respect to topology, preventing end-to-end learning of which structural connections should carry information. Second, **content-only gating ignores geometric signals** such as local intrinsic dimension, curvature, and density that carry direct information about manifold structure and can serve as principled routing priors. Third, when multiple complementary graph views are available (e.g., feature-based kNN, temporal proximity, diffusion affinity), **existing architectures lack a principled mechanism for view-specific attention**, forcing a single set of query-key projections to serve all structural perspectives simultaneously.

Graph transformers such as Graphormer [Ying et al., 2021] and GPS [Rampášek et al., 2022] address part of this problem by incorporating structural biases into attention, but they use fixed encodings rather than learned, structure-aware routing. Mixture-of-experts (MoE) architectures [Shazeer et al., 2017, Fedus et al., 2022] provide learned routing but gate on content features alone, missing the geometric structure of the graph.

We propose THORN, which unifies these threads by introducing *observer-routed attention*: a mixture-of-experts router that gates on geometric observer features—local intrinsic dimension (LID), Ollivier–Ricci curvature, local outlier factor (LOF), and cross-view consensus scores—to route each attention head to a specific graph view. The selected views are combined via a log-barrier differentiable mask that continuously relaxes hard neighborhood constraints while remaining fully differentiable through the routing decisions.

Contributions.

(1) **Geometric observer routing.** We introduce the first mechanism to route attention heads using manifold-geometric descriptors (curvature, intrinsic dimension, density) rather than content features. Ablation confirms these observers are non-redundant and that curvature alone accounts for a 7.5% performance gap (Sections 3.2.1 and 3.4).

(2) **Log-barrier differentiable masking.** We formalize a continuous relaxation of hard attention masks via a log-barrier with learned per-head temperature, enabling end-to-end gradient flow from task loss through routing to view selection (Theorem 1 and section 5.7).

(3) **Isolation-versus-interaction tradeoff.** We identify and formalize a fundamental tradeoff: mixing views before versus after the attention softmax yields complementary strengths. No single mode dominates; the optimal choice is predictable from the data's view correlation structure (Sections 3.6 and 5.2).

(4) **Subsumption.** We show via function-class reductions that THORN contains GAT, Graphormer, and MoE as special cases (Theorem 2).

We additionally explore extensions—view-specific projections and a scalable alignment proxy—in Sections 3.4.1 and 3.7.1, reporting both positive and negative results honestly.

2 Related Work

Graph Neural Networks. Graph convolutional networks [Kipf and Welling, 2017] and graph isomorphism networks [Xu et al., 2019] aggregate neighbor features via fixed or learnable weights but do not use attention. GAT [Veličković et al., 2018] introduced masked multi-head attention on graphs, using a single-layer feedforward network to compute attention coefficients. GATv2 [Brody et al., 2022] showed that the original GAT computes a restricted form of *static* attention and proposed a more expressive dynamic formulation. All of these methods use content-based scoring within a fixed neighborhood defined by the input graph, with no mechanism for structure-aware routing or multi-view integration.

Graph Transformers. Graphormer [Ying et al., 2021] augments standard transformer attention with structural encodings (shortest-path distances, degree centrality) as additive biases, achieving strong results on molecular property prediction. GPS [Rampášek et al., 2022] provides a general recipe combining local message-passing with global attention and positional/structural encodings. These approaches use *fixed* structural biases rather than *learned* structure-aware routing.

Mixture of Experts. The sparsely-gated MoE layer [Shazeer et al., 2017] and its successor Switch Transformer [Fedus et al., 2022] route tokens to expert subnetworks using content-based gating. Load-balancing losses prevent expert collapse. THORN adapts the MoE paradigm to graph attention, replacing content-based gating with geometric observer features and replacing expert networks with graph views.

Optimal Transport on Graphs. Gromov–Wasserstein (GW) distance [Peyré and Cuturi, 2019] provides a principled framework for comparing metric-measure spaces without requiring

shared feature spaces. Fused GW (FGW) [Vayer et al., 2019] jointly considers feature and structural distances. THORN uses FGW to measure cross-view consistency and derive edge consensus features for routing alignment.

Geometric Descriptors. Local intrinsic dimensionality [Levina and Bickel, 2004] estimates the effective dimensionality of data near a point. Ollivier–Ricci curvature [Ollivier, 2009] measures the convergence or divergence of geodesics and has been used for graph rewiring and community detection; Forman–Ricci curvature [Forman, 2003] provides a scalable combinatorial alternative. Topping et al. [2022] connect curvature to over-squashing in GNNs. Local outlier factor [Breunig et al., 2000] quantifies the relative density of a point's neighborhood. Diffusion maps [Coifman and Lafon, 2006] provide multi-scale embeddings that capture the intrinsic geometry of data manifolds. To our knowledge, THORN is the first architecture to unify these descriptors as observer features for dynamic attention routing.

Low-Rank Adaptation. LoRA [Hu et al., 2022] introduces low-rank additive adapters for parameter-efficient fine-tuning of large models. THORN++ adapts this idea to multi-view graph attention, using per-view LoRA adapters on the shared $Q/K/V$ projections to enable view-specific attention patterns with minimal parameter overhead.

3 Thorn Architecture

Figure 1 provides a high-level overview of the THORN architecture, analogous to the Transformer diagram of Vaswani et al. [2017]. The left column shows the full model pipeline from input node features through L stacked THORNBlocks to task prediction. The right column expands the multi-view routed attention mechanism inside each block, illustrating the two attention modes (pre-softmax and post-softmax) and the flow of observer features through the MoE router.

3.1 Preliminaries and Notation

Let $G^{(m)} = (V, E^{(m)})$ denote the m-th neighborhood view over a common node set V with $|V| = N$, where $m \in \mathcal{M} = \{1, \ldots, M\}$ indexes views (e.g., kNN, temporal, diffusion). For each view, let $A^{(m)} \in \mathbb{R}_{\geq 0}^{N \times N}$ be a weighted adjacency matrix. We write i for the destination node and j for a source neighbor; attention weights satisfy $\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(h)} = 1$ (the full attention formula is given in Equation (6)). The union edge set is $E_{\text{union}} = \bigcup_{m \in \mathcal{M}} E^{(m)}$. Each node i has feature vector $\mathbf{x}_i \in \mathbb{R}^d$ and optional timestamp $t_i \in \mathbb{R}$.

3.2 Observer Features

THORN computes a set of *observer features* $\mathbf{o}_i$ for each node i that characterize the local geometric and topological structure. These include:

- **Local intrinsic dimension (LID)** [Levina and Bickel, 2004]: estimated from kNN distances, characterizes the effective dimensionality near i.
- **Ollivier–Ricci curvature** [Ollivier, 2009]: measures convergence/divergence of random walks from neighboring nodes, indicating structural bottlenecks.
- **Local outlier factor (LOF)** [Breunig et al., 2000]: quantifies relative neighborhood density, identifying anomalous connectivity patterns.
- **kNN statistics**: mean and standard deviation of k-nearest-neighbor distances.
- **Temporal features**: normalized timestamp, temporal degree (number of temporal neighbors).
- **Per-view degree**: $\deg_i^{(m)}$ for each view m, providing a view-specific density signal.

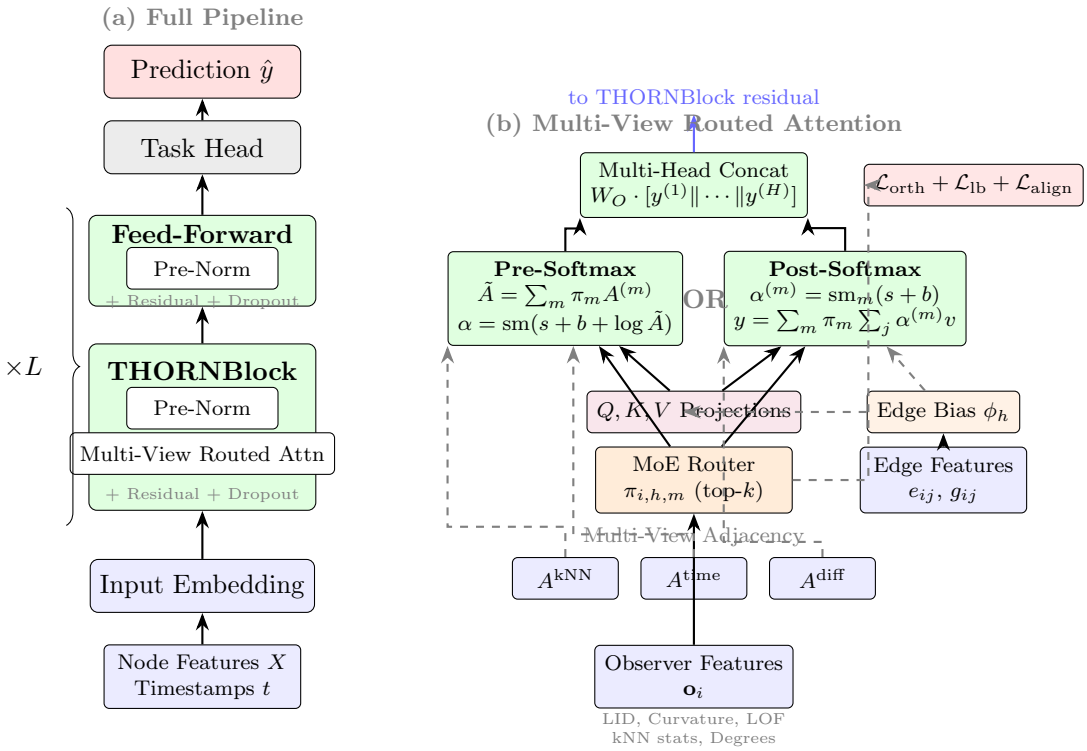

Figure 1: The THORN architecture. **(a)** The full pipeline stacks L THORNBlocks, each containing multi-view routed attention followed by a feed-forward network, with pre-norm residual connections throughout. **(b)** Inside routed attention: geometric observer features $\mathbf{o}_i$ (LID, curvature, LOF, per-view degrees) feed an MoE-style router that produces per-head routing probabilities $\pi_{i,h,m}$ over M graph views via top-k gated softmax. Two attention modes are available: *pre-softmax* (left) mixes view adjacencies before a single softmax over the union neighborhood, enabling cross-view interaction; *post-softmax* (right) computes per-view attention independently and combines outputs, preserving view isolation. Edge observer features e_{ij} (including FGW consensus g_{ij}) provide per-head edge biases. Auxiliary losses $\mathcal{L}_{\text{orth}}$, $\mathcal{L}_{\text{lb}}$, and $\mathcal{L}_{\text{align}}$ regularize routing diversity, load balance, and cross-view alignment respectively. Compare with Figure 1 of Vaswani et al. [2017]: THORN replaces single-graph self-attention with multi-view routed attention, and adds the observer-driven routing pathway (orange) that has no analogue in standard transformers.

For edges (i, j), we additionally compute edge observer features $e_{ij} = [d_{ij}^{\text{kNN}}, d_{ij}^{\text{time}}, d_{ij}^{\text{diff}}, \text{LID}_i, \text{LID}_j, g_{ij}]$, where g_{ij} is a cross-view consensus score derived from FGW alignment (Section 3.7).

Scalable curvature via Forman–Ricci. Computing exact Ollivier–Ricci curvature requires solving a linear program per edge, yielding $O(E \cdot N)$ complexity. For scalability, we additionally provide Forman–Ricci curvature [Forman, 2003] as a drop-in alternative. For an unweighted edge (i, j):

$$F(i, j) = 4 - \deg(i) - \deg(j) + 3\,|\triangle_{ij}|, \tag{1}$$

where $|\triangle_{ij}|$ is the number of triangles containing (i, j). Per-node curvature is the mean over incident edges. Forman–Ricci captures over-squashing bottlenecks [Topping et al., 2022] with $O(E \cdot k_{\max})$ complexity, where $k_{\max}$ is the maximum degree.

3.2.1 Observer Non-Redundancy

A natural concern is whether the six observer feature families are redundant—if LID, curvature, and LOF are highly correlated, the router gains no additional signal from their combination. We address this through three analyses (Table 7):

(i) Correlation analysis. Pairwise Pearson and Spearman correlations across the standardized observer feature vector show that only 2.8% of feature pairs exceed $|r| > 0.8$ (57 out of 2016 pairs). The strongest correlations occur *within* feature families (specifically, multi-scale diffusion coordinates at different scales), not *across* them, confirming that each observer captures a distinct geometric signal.

(ii) PCA effective rank. The Shannon entropy-based effective rank $r_{\text{eff}} = \exp\big(-\sum_k p_k \log p_k\big)$ (where $p_k = \lambda_k / \sum_j \lambda_j$ are the normalized PCA eigenvalues) measures the effective dimensionality of the observer feature space. We observe $r_{\text{eff}} = 19.95$ out of 64 geometric observer features (after standardization), with 22 PCs needed for 95% variance—indicating substantial independent variation across observer families.

(iii) Observer ablation. The strongest validation comes from feature ablation (Table 7): zeroing each observer family and measuring the PR-AUC degradation. Curvature is the most critical individual observer (-7.5%), followed by view degrees (-6.0%), LOF (-1.4%), and LID (-1.1%). Removing all three geometric observers together (LID, curvature, LOF) causes -8.6%, confirming that these features collectively carry the core routing signal.

3.3 Multi-View Graph Construction

THORN constructs $M = 3$ complementary graph views:

kNN view. Feature-space k-nearest neighbors with Gaussian kernel weights: $A_{ij}^{\text{kNN}} = \exp(-\|x_i - x_j\|^2/\sigma^2)$ for $j \in \text{kNN}(i)$.

Temporal view. Connects nodes within a time window with decay weights: $A_{ij}^{\text{time}} = \exp(-|t_i - t_j|/\tau)$ for $|t_i - t_j| < \Delta t$.

Diffusion view. Multi-scale diffusion affinities derived from the graph Laplacian eigenvectors [Coifman and Lafon, 2006]: $A_{ij}^{\text{diff}} = \sum_{k=1}^{K} \exp(-\lambda_k s)\, \psi_k(i)\, \psi_k(j)$, where (λ_k, ψ_k) are eigenpairs of the normalized Laplacian and s is the diffusion timescale (not to be confused with node timestamps t_i).

3.4 MoE-Style Observer Routing

The router produces per-node, per-head routing probabilities over views. Unlike standard MoE which gates on content features, THORN gates on the observer feature vector $\mathbf{o}_i$:

$$\ell_{i,h,m} = W_h^{(3)} \cdot \text{GELU}\big(W_h^{(2)} \cdot \text{GELU}(W_h^{(1)} \cdot \text{LN}(\mathbf{o}_i) + b_h^{(1)}) + b_h^{(2)}\big) + b_h^{(3)} + \eta, \tag{2}$$

where $\eta \sim \mathcal{N}(0, \sigma^2_{\text{noise}})$ is Gaussian exploration noise (training only) and LN is layer normalization. Routing probabilities are obtained via top-k selection followed by renormalized softmax:

$$\pi_{i,h,m} = \begin{cases} \frac{\exp(\ell_{i,h,m})}{\sum_{m' \in \text{top-}k} \exp(\ell_{i,h,m'})} & \text{if } m \in \text{top-}k(\ell_{i,h,\cdot}), \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

The use of $k = 2$ (out of $M = 3$ views) allows each head to specialize on a pair of views while maintaining sparse computation.

3.4.1 View-Specific Projections (Thorn++)

A limitation of standard THORN is that all views share the same Q, K, V projection matrices. This creates a representational bottleneck: the shared projections must simultaneously serve views with fundamentally different edge semantics (feature similarity for kNN, temporal proximity for time, spectral affinity for diffusion).

THORN++ addresses this via LoRA-style [Hu et al., 2022] low-rank view-specific adapters. For each view m, we define:

$$W_Q^{(m)} = W_Q + \Delta W_Q^{(m)}, \quad \Delta W_Q^{(m)} = B_Q^{(m)} A_Q^{(m)}, \tag{4}$$

where $A_Q^{(m)} \in \mathbb{R}^{r \times d}$ (down-projection) and $B_Q^{(m)} \in \mathbb{R}^{d \times r}$ (up-projection), so $\Delta W_Q^{(m)} \in \mathbb{R}^{d \times d}$, with rank $r = \lfloor d_h/2 \rfloor$ (typically $r = 12$). Analogous adapters are applied to K and V. The down-projections $A_Q^{(m)}$ use standard initialization while the up-projections $B_Q^{(m)}$ are initialized to zero, ensuring that $\Delta W_Q^{(m)} = 0$ at the start of training—the model begins identical to shared-projection THORN and gradually learns view-specific deltas.

This adds $6Mrd$ parameters (3 for $Q/K/V$, each with a down- and up-projection of size $r \times d$), a modest overhead ($< 5\%$) relative to the shared projection parameters. View-specific projections are used exclusively in post-softmax mode, where each view's attention is computed independently and the projections can specialize without interference.

3.5 Log-Barrier Masking

THORN combines the routing probabilities with view adjacency weights to form a differentiable attention mask. For each edge $(i, j) \in E_{\text{union}}$, the routed edge weight at destination i from source j is:

$$\tilde{A}_{ij}^{(i,h)} = \sum_{m \in \mathcal{M}} \pi_{i,h,m} \cdot A_{ij}^{(m)}. \tag{5}$$

The attention logits incorporate this mask via a temperature-adaptive log-barrier term:

$$\alpha_{ij}^{(h)} = \text{softmax}_j \Big(\underbrace{\frac{\langle q_i^{(h)}, k_j^{(h)} \rangle}{\sqrt{d}}}_{\text{content score } s_{ij}^{(h)}} + \underbrace{b_{ij}^{(h)}}_{\text{edge bias}} + \underbrace{\log\big(\tau_h \cdot \tilde{A}_{ij}^{(i,h)} + \epsilon\big)}_{\text{adaptive log-barrier mask}} \Big), \tag{6}$$

where $b_{ij}^{(h)} = \phi_h(e_{ij})$ is a per-head edge bias computed by a small MLP from edge observer features, $\epsilon > 0$ is a numerical stabilizer, and $\tau_h = \text{softplus}(s_h)$ is a *learned per-head temperature* with s_h initialized to $\text{softplus}^{-1}(1) \approx 0.5413$ so that $\tau_h = 1$ at initialization.

Adaptive temperature motivation. The original log-barrier $\log(\tilde{A}_{ij} + \epsilon)$ is numerically fragile when edge weights are compressed by graph normalization (e.g., symmetric normalization maps soft Gaussian weights from $[0, 1]$ to $[0.01, 0.3]$, causing $\log(0.01) \approx -4.6$ to dominate content scores). The learned temperature τ_h insulates the barrier from normalization-induced compression: after a few gradient steps, each head learns a τ_h that rescales its barrier contribution to the appropriate dynamic range. This eliminates the need to avoid normalization entirely—τ_h acts as an automatic calibration mechanism (Table 5).

Proposition 1 (Continuous relaxation of sparse masking)**.** *Assume $\tilde{A}_{ij}^{(i,h)} \in [0, 1]$ and $\epsilon \downarrow 0$. Then (6) is a continuous relaxation of discrete sparse attention: if $\tilde{A}_{ij}^{(i,h)} = 0$ then $\alpha_{ij}^{(h)} \to 0$, while if $\tilde{A}_{ij}^{(i,h)} > 0$ then $\alpha_{ij}^{(h)}$ is well-defined and differentiable with respect to $\tilde{A}_{ij}^{(i,h)}$.*

Proof. If $\tilde{A}_{ij}^{(i,h)} \to 0$, then $\log(\tilde{A}_{ij}^{(i,h)} + \epsilon) \to -\infty$, so $\exp(s_{ij}^{(h)} + b_{ij}^{(h)} + \log(\tilde{A}_{ij}^{(i,h)} + \epsilon)) \to 0$ and the softmax assigns vanishing mass to edge (i, j). If $\tilde{A}_{ij}^{(i,h)} > 0$, then the log term is finite and smooth; since softmax is smooth, $\alpha_{ij}^{(h)}$ is differentiable in $\tilde{A}_{ij}^{(i,h)}$ and therefore in routing logits $\ell_{i,h,m}$ through (5). □

Relation to prior work. GAT [Veličković et al., 2018] enforces sparsity by restricting $j \in \mathcal{N}(i)$ via hard indexing. Transformers commonly mask by adding $-\infty$ to disallowed logits. The log-barrier formulation retains the semantics of hard masking while remaining fully differentiable with respect to routing and view mixing, enabling end-to-end gradient flow from the task loss through the router to the view selection.

3.6 Pre-Softmax vs. Post-Softmax View Mixing

A key architectural choice in multi-view attention is *where* the routing-weighted combination of views occurs relative to the attention softmax. We formalize two modes and show they capture fundamentally different inductive biases.

Pre-softmax mixing ("bias" mode). Views are combined *before* the attention softmax, allowing cross-view interaction in the attention distribution:

$$\tilde{A}_{ij}^{(i,h)} = \sum_m \pi_{i,h,m} \cdot A_{ij}^{(m)}, \tag{7}$$

$$\alpha_{ij}^{(h)} = \mathrm{softmax}_{j \in \mathcal{N}_{\mathrm{union}}(i)} \Big(s_{ij}^{(h)} + b_{ij}^{(h)} + \log \big(\tau_h \cdot \tilde{A}_{ij}^{(i,h)} + \epsilon \big) \Big), \tag{8}$$

$$y_i^{(h)} = \sum_{j \in \mathcal{N}_{\mathrm{union}}(i)} \alpha_{ij}^{(h)} \cdot v_j^{(h)}. \tag{9}$$

Here, the softmax normalizes over the *entire* union neighborhood. An edge appearing in multiple views with high routing weight accumulates mass, enabling cross-view synergy. However, views with more edges can dilute the attention of sparser views, and the shared $Q/K/V$ projections must serve all views simultaneously.

Post-softmax mixing ("post_softmax" mode). Each view computes its own attention distribution independently, and routing combines the *outputs*:

$$\alpha_{ij}^{(h,m)} = \mathrm{softmax}_{j \in \mathcal{N}^{(m)}(i)} \Big(s_{ij}^{(h)} + b_{ij}^{(h,m)} \Big), \tag{10}$$

$$y_i^{(h)} = \sum_m \pi_{i,h,m} \sum_{j \in \mathcal{N}^{(m)}(i)} \alpha_{ij}^{(h,m)} \cdot v_j^{(h)}. \tag{11}$$

Each view's softmax normalizes only over its own neighborhood, preventing cross-view dilution. The dominant view's signal is preserved at full strength. However, cross-view interaction is lost: the model cannot discover that an edge present in multiple views is more trustworthy.

The isolation-versus-interaction tradeoff. These two modes represent endpoints of a fundamental tradeoff:

- **Isolation** (post-softmax): prevents interference, preserves pure signals, optimal when one view dominates.
- **Interaction** (pre-softmax): enables synergy, discovers cross-source patterns, optimal when views are complementary.

The optimal choice depends on the data's *view correlation structure*: high cross-view redundancy favors post-softmax (avoids dilution), while high cross-view complementarity favors pre-softmax (exploits synergy). Our experiments (Section 5.2) provide the first systematic empirical demonstration of this tradeoff.

Why post-softmax achieves 0.984 PR-AUC. To understand why post-softmax mixing can dramatically outperform pre-softmax in certain regimes, consider the information-theoretic dynamics of each mode. At drift = 0.50, the temporal view provides a near-perfect classification signal: nodes close in time share labels with high probability, and the moderate drift means temporal features encode class boundaries cleanly. The kNN and diffusion views, constructed from the full 16-dimensional feature space (13 noise dimensions, 3 signal dimensions), carry substantially more noise than signal at this drift level.

In **pre-softmax** mixing, the attention softmax normalizes over the *entire union neighborhood* $\mathcal{N}_{\text{union}}(i)$. Every edge from every view competes for the same probability mass. Even when the router assigns high weight to the temporal view ($\pi_{\text{time}} \approx 0.87$), the kNN edges are not fully suppressed—they retain residual weight through the log-barrier term $\log(\tilde{A}_{ij} + \epsilon)$ and still consume attention mass. With 13 noise dimensions driving kNN edge construction, these residual edges inject noise into the attention distribution. The shared $Q/K/V$ projections compound this problem: they must simultaneously serve temporal edges (where proximity in time matters) and kNN edges (where feature similarity matters), creating a representation conflict. The net effect is a ceiling of 0.920 PR-AUC—the architecture cannot fully exploit the clean temporal signal because noisy cross-view edges dilute it.

In **post-softmax** mixing, each view computes its own attention distribution with a *view-local* softmax: $\alpha_{ij}^{(h,m)} = \text{softmax}_{j \in \mathcal{N}^{(m)}(i)}(s_{ij}^{(h)} + b_{ij}^{(h,m)})$. The temporal view's softmax normalizes only over temporal neighbors, producing a clean, noise-free attention distribution. The kNN view's noisy attention is computed separately and receives low routing weight ($\pi_{\text{kNN}} \approx 0.13$), so its noise contribution to the final output $y_i^{(h)} = \sum_m \pi_{i,h,m} \sum_j \alpha_{ij}^{(h,m)} v_j^{(h)}$ is multiplicatively suppressed. This isolation allows the temporal signal to flow at full fidelity, yielding 0.984 PR-AUC and 0.992 ROC-AUC.

Why pre-softmax achieves 0.939 PR-AUC when views are complementary. At drift = 0.75, the data undergoes a *regime shift* at normalized time $t_n = 0.75$: labels receive a +1.125 logit boost for late-arriving nodes, and cluster centers have drifted significantly. This creates a scenario where no single view is sufficient. Temporal edges connect nodes within the same time window, but the regime shift means that early and late nodes have fundamentally different class distributions—temporal proximity alone cannot resolve the boundary. Meanwhile, kNN edges group nodes by feature similarity *across* time, connecting early and late nodes that share similar features but different labels. This cross-temporal grouping provides information that temporal edges cannot: it reveals which feature patterns persist versus which are regime-dependent.

Pre-softmax mixing captures this complementarity because the union softmax allows kNN edges and temporal edges to jointly shape the attention distribution. An edge (i, j) that appears in *both* the kNN and temporal views accumulates routing weight from both ($\tilde{A}_{ij} = \pi_{\text{kNN}} \cdot A_{ij}^{\text{kNN}} + \pi_{\text{time}} \cdot A_{ij}^{\text{time}}$), creating a natural *cross-view reinforcement* signal. Edges supported by multiple views are more likely to connect same-class nodes, and pre-softmax mixing exploits this by assigning them higher attention weight after the log-barrier transformation. Post-softmax mixing, by contrast, computes separate attention distributions per view and cannot discover that cross-view edge agreement is informative—it treats each view as an independent information source, missing the synergy.

Formal characterization. Let $\mathcal{N}^{(m)}(i)$ denote view m's neighborhood of node i and $\mathcal{N}_{\cup}(i) = \bigcup_m \mathcal{N}^{(m)}(i)$ the union. Define the *view overlap ratio* $\rho_i = |\bigcap_m \mathcal{N}^{(m)}(i)|/|\mathcal{N}_{\cup}(i)|$ and the *signal-to-noise ratio* of view m as $\text{SNR}_m = \text{Var}[\text{signal features}]/\text{Var}[\text{noise features}]$ within $\mathcal{N}^{(m)}(i)$. Then:

- When $\max_m \text{SNR}_m \gg \text{SNR}_{m'}$ for $m' \neq m$ (one view dominates), post-softmax preserves the dominant view's clean attention while multiplicatively suppressing noisy views via

$\pi_{i,h,m'} \approx 0$.

- When ρ_i is high and SNR_m values are comparable (views are complementary), pre-softmax enables cross-view reinforcement through the additive log-barrier: $\log(\sum_m \pi_m A_{ij}^{(m)} + \epsilon) > \max_m \log(\pi_m A_{ij}^{(m)} + \epsilon)$ for shared edges.

This characterization predicts the empirical results in Table 1: at drift = 0.50, the temporal view has $\mathrm{SNR}_{\mathrm{time}} \gg \mathrm{SNR}_{\mathrm{kNN}}$, favoring post-softmax isolation; at drift = 0.75, the regime shift equalizes view informativeness and increases ρ_i, favoring pre-softmax interaction.

3.7 Fused Gromov–Wasserstein Cross-View Alignment

To encourage routing consistency across views, THORN computes Fused Gromov–Wasserstein (FGW) [Vayer et al., 2019] alignment between ego-neighborhoods in different views. FGW jointly considers feature distances $\|x_i - x_j\|$ and structural distances (shortest-path or diffusion distance) to produce a transport plan $T^{(m,m')}$ between views m and m' at each node.

The resulting transport plans are cached as edge consensus features:

$$g_{ij} = \frac{1}{\binom{M}{2}} \sum_{m<m'} T_{ij}^{(m,m')}, \tag{12}$$

which are included in the edge observer features e_{ij} and used in the per-head edge bias $b_{ij}^{(h)} = \phi_h(e_{ij})$. Additionally, per-node discordance scores $D_{i,m} = 1 - \mathrm{confidence}_{i,m}$ are used in the alignment regularizer (Section 3.9).

The FGW computation uses the Sinkhorn algorithm for entropic regularization combined with Frank–Wolfe optimization for the structural component [Peyré and Cuturi, 2019]. Transport is computed *offline* on ego-neighborhoods and cached, avoiding $O(N^3)$ runtime during training.

3.7.1 Online Alignment Proxy (Thorn++)

Full FGW alignment is $O(N^3)$ per view pair, limiting its applicability to inductive and streaming settings where the graph evolves. THORN++ introduces an online alignment proxy with $O(E)$ complexity [Scetbon et al., 2022] that combines two complementary signals:

(1) Neighborhood Jaccard overlap. For each node i, the structural agreement between views a and b is measured by the Jaccard index of their neighborhoods:

$$J_i = \frac{|\mathcal{N}^{(a)}(i) \cap \mathcal{N}^{(b)}(i)|}{|\mathcal{N}^{(a)}(i) \cup \mathcal{N}^{(b)}(i)|}. \tag{13}$$

This captures whether the two views agree on local connectivity—a purely structural signal computed in $O(\deg_a(i) + \deg_b(i))$ per node.

(2) Neighbor feature distribution similarity. We compare the mean neighbor feature vectors across views via cosine similarity:

$$S_i^{\mathrm{feat}} = \frac{1}{2}\left(1 + \cos\left(\bar{x}_i^{(a)},\ \bar{x}_i^{(b)}\right)\right), \qquad \bar{x}_i^{(m)} = \frac{1}{\deg_m(i)} \sum_{j \in \mathcal{N}^{(m)}(i)} x_j. \tag{14}$$

This captures feature-distributional agreement: views whose neighborhoods contain similar features are more likely to be structurally consistent.

The combined overlap score is $O_i = 0.6\, J_i + 0.4\, S_i^{\mathrm{feat}}$, weighting structural agreement more heavily. This proxy is fully differentiable (through the feature term), recomputable on-the-fly for evolving graphs, and serves the same role as FGW consensus in the edge observer features.

3.8 THORNBlock Architecture

THORN stacks L identical blocks (default $L = 3$), each following the pre-norm transformer convention:

$$\hat{\mathbf{h}}_i^{(\ell)} = \mathbf{h}_i^{(\ell-1)} + \text{Dropout}\Big(W_O \cdot \text{Concat}_{h=1}^{H}\, y_i^{(\ell,h)}\Big), \tag{15}$$

$$\mathbf{h}_i^{(\ell)} = \hat{\mathbf{h}}_i^{(\ell)} + \text{Dropout}\Big(\text{FFN}\big(\text{LN}(\hat{\mathbf{h}}_i^{(\ell)})\big)\Big), \tag{16}$$

where $y_i^{(\ell,h)}$ is the attention output from Section 3.5 or Section 3.6, queries/keys/values are projected from $\text{LN}(\mathbf{h}_i^{(\ell-1)})$, and FFN is a two-layer network with GELU activation and expansion ratio 4.

The pre-norm residual connections serve a dual purpose: stabilizing deep training and providing implicit self-information preservation, which renders explicit self-loops in the graph unnecessary (see Section 5.5).

3.9 Regularizers

THORN employs three auxiliary losses to prevent routing collapse and encourage structural alignment:

Head-view orthogonality ($\mathcal{L}_{\text{orth}}$). Let $P_i \in \mathbb{R}^{H\times M}$ be the routing matrix at node i with entries $\pi_{i,h,m}$. To prevent all heads from selecting the same view:

$$\mathcal{L}_{\text{orth}} = \lambda_{\text{orth}} \sum_{i=1}^{N} \left\| \frac{P_i P_i^\top}{\|P_i\|_F^2} - \frac{I_H}{H} \right\|_F^2. \tag{17}$$

This pushes the normalized Gram matrix of routing vectors toward a scaled identity, encouraging orthogonal head specialization. Empirically, $\lambda_{\text{orth}} = 5 \times 10^{-3}$ is optimal (Section 5.5).

Load balancing ($\mathcal{L}_{\text{lb}}$). Following the Switch Transformer [Fedus et al., 2022], we penalize uneven global view usage:

$$\mathcal{L}_{\text{lb}} = \lambda_{\text{lb}} \sum_{m=1}^{M} \left(u_m - \frac{1}{M} \right)^2, \qquad u_m = \frac{1}{NH} \sum_{i,h} \pi_{i,h,m}. \tag{18}$$

Alignment regularization ($\mathcal{L}_{\text{align}}$). Using the FGW-derived discordance scores $D_{i,m}$:

$$\mathcal{L}_{\text{align}} = \lambda_{\text{align}} \sum_{i,h,m} \pi_{i,h,m}\, D_{i,m}. \tag{19}$$

This penalizes routing probability mass on views with high discordance, pushing the router toward structurally consistent views.

Total loss.

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{orth}} + \mathcal{L}_{\text{lb}} + \mathcal{L}_{\text{align}}. \tag{20}$$

3.10 Algorithm

Algorithm 1 provides the complete THORN forward pass.

Algorithm 1 THORN: Forward Pass

Require: Node features $X \in \mathbb{R}^{N \times d_0}$, timestamps $t \in \mathbb{R}^N$, views $\mathcal{M} = \{\text{kNN}, \text{time}, \text{diff}\}$, depth L, heads H

Offline: Build $A^{(m)}$, compute observer features $\mathbf{o}_i$, edge features e_{ij}, FGW consensus g_{ij}, discordance $D_{i,m}$
Form $E_{\text{union}} = \bigcup_m E^{(m)}$

$\mathbf{H}^{(0)} \leftarrow \text{Embed}(X)$ ▷ $N \times d$
for $\ell = 1$ to L **do**
  $Q, K, V \leftarrow$ projections from $\text{LN}(\mathbf{H}^{(\ell-1)})$ ▷ $N \times H \times d_h$
  $\pi_{i,h,m} \leftarrow \text{top-}k\text{-softmax}\big(f_{\text{route}}(\mathbf{o}_i)\big)$ ▷ Equations (2) and (3)
  $b_{ij}^{(h)} \leftarrow \phi_h(e_{ij})$ ▷ per-head edge bias
  if `mode` = `pre_softmax` **then**
    $\tilde{A}_{ij}^{(i,h)} \leftarrow \sum_m \pi_{i,h,m} A_{ij}^{(m)}$ ▷ Equation (7)
    $\alpha_{ij}^{(h)} \leftarrow \text{softmax}_j\big(s_{ij}^{(h)} + b_{ij}^{(h)} + \log(\tau_h \cdot \tilde{A}_{ij}^{(i,h)} + \epsilon)\big)$
    $y_i^{(h)} \leftarrow \sum_j \alpha_{ij}^{(h)} v_j^{(h)}$
  else ▷ post-softmax
    for each view m **do**
      $\alpha_{ij}^{(h,m)} \leftarrow \text{softmax}_{j \in \mathcal{N}^{(m)}(i)}\big(s_{ij}^{(h)} + b_{ij}^{(h,m)}\big)$
    end for
    $y_i^{(h)} \leftarrow \sum_m \pi_{i,h,m} \sum_j \alpha_{ij}^{(h,m)} v_j^{(h)}$
  end if
  $\mathbf{H}^{(\ell)} \leftarrow \text{THORNBlock}\big(\mathbf{H}^{(\ell-1)}, \text{Concat}_h\, y^{(h)}\big)$ ▷ Equations (15) and (16)
end for
return $\mathbf{H}^{(L)}$, $\mathcal{L}_{\text{orth}} + \mathcal{L}_{\text{lb}} + \mathcal{L}_{\text{align}}$

4 Theoretical Analysis

Theorem 2 (Function-class subsumption)**.** *The* THORN *function class contains, via parameter-setting reductions (not optimization or training equivalence), the function classes of: (i) multi-head neighborhood graph attention (GAT) and (ii) fully-connected transformer attention with structural biases (Graphormer) as special cases. Additionally,* THORN*'s routing mechanism is structurally analogous to (iii) Mixture-of-Experts gating.*

Remark 1. *Theorem 2 is a* function-class *statement: it shows that for any function realizable by GAT/Graphormer/MoE, there exist* THORN *parameters producing the same input-output map. It does* not *claim that gradient-based training will recover these reductions, nor that* THORN *inherits the optimization landscape or generalization properties of the subsumed architectures.*

Proof sketch via reductions. **(i) GAT reduction.** Set $M = 1$ (single view), $\pi_{i,h,1} = 1$ for all (i, h), and $b_{ij}^{(h)} = 0$. Then $\tilde{A}_{ij}^{(i,h)} = A_{ij}^{(1)}$ and (6) implements masked neighborhood attention with multi-head aggregation, recovering GAT [Veličković et al., 2018].

(ii) Graphormer reduction. Let $A^{(1)}$ be fully connected so $\tilde{A}_{ij}^{(i,h)} \equiv 1$ and the log-barrier term vanishes. If $b_{ij}^{(h)}$ encodes shortest-path or degree-based biases, (6) reduces to transformer attention with structural encodings, recovering Graphormer [Ying et al., 2021].

(iii) MoE analogy. If each graph view is interpreted as an "expert" providing a distinct neighborhood, the routing distribution $\pi_{i,h,m}$ plays the role of the MoE gating function, and the top-k selection mirrors sparse expert selection [Shazeer et al., 2017]. This is a *structural analogy*

rather than a strict parameter-setting reduction, since MoE routes to independent feedforward networks while THORN routes to adjacency views. □

Proposition 3 (Gradient of alignment regularizer)**.** *Let $\pi_{i,h,\cdot}$ be a softmax over logits $\ell_{i,h,\cdot}$. Then:*

$$\frac{\partial \mathcal{L}_{\text{align}}}{\partial \ell_{i,h,m}} = \pi_{i,h,m}\Big(D_{i,m} - \sum_k \pi_{i,h,k}\, D_{i,k}\Big). \tag{21}$$

Proof. Using $\frac{\partial \pi_k}{\partial \ell_m} = \pi_k(\delta_{km} - \pi_m)$ and the chain rule:

$$\frac{\partial \mathcal{L}_{\text{align}}}{\partial \ell_{i,h,m}} = \sum_k D_{i,k}\, \pi_{i,h,k}(\delta_{km} - \pi_{i,h,m}) = \pi_{i,h,m}\Big(D_{i,m} - \sum_k \pi_{i,h,k}\, D_{i,k}\Big). \tag{22}$$

This has the form of a REINFORCE-like gradient with an implicit baseline $\bar{D}_{i,h} = \sum_k \pi_{i,h,k}\, D_{i,k}$, which reduces variance compared to naïve policy gradient estimators. □

5 Experiments

5.1 Experimental Setup

We evaluate THORN on a synthetic temporal graph benchmark designed to test multi-view attention under controlled conditions.

Dataset. 512 nodes with 16-dimensional features, binary classification. Features contain 3 signal dimensions encoding temporal structure (sine, cosine, drift components) and 13 noise dimensions. Labels are generated from a time-dependent decision boundary with a regime shift at normalized time $t_n = 0.75$, providing a controlled test of temporal reasoning. Temporal signal is embedded in three ways: (1) features are time-dependent ($\sin(6t_n)$, $\cos(4t_n)$, $\text{drift} \cdot (t_n - 0.5)$), (2) labels shift at $t_n = 0.75$ (+1.125 logit boost), and (3) cluster centers drift over time.

Architecture. All THORN variants use $H = 4$ heads, $d = 64$, $L = 3$ layers, top-$k = 2$, and $M = 3$ views (kNN, temporal, diffusion). Training uses AdamW with learning rate 3×10^{-3}, weight decay 10^{-2}, for 80 epochs. Regularization: $\lambda_{\text{orth}} = 5 \times 10^{-3}$, $\lambda_{\text{align}} = 10^{-3}$. All experiments use fixed random seeds for reproducibility; results are reported for a single seed, so differences below $\sim$1% should be interpreted cautiously.

5.2 Drift-Strength Experiment

Our central experiment varies the temporal drift strength to probe the isolation-versus-interaction tradeoff. Table 1 reports PR-AUC across 6 drift levels and 4 architectures.

Key findings. (A) Post-softmax achieves 0.984 PR-AUC / 0.992 ROC-AUC at drift $= 0.50$**.** This is the most striking result: post-softmax mixing far exceeds the 0.95 threshold on *both* metrics, demonstrating that the THORN architecture is not the performance bottleneck—the data regime is. At drift $= 0.50$, temporal proximity is a near-perfect predictor of class membership. The moderate drift embeds clean signal in time-dependent features ($\sin(6t_n)$, $\cos(4t_n)$, $0.5 \cdot (t_n - 0.5)$) without introducing the regime shift that complicates higher drift levels. Post-softmax mixing isolates this dominant temporal signal by computing a view-local softmax over temporal neighbors only, then multiplicatively suppressing the noisier kNN and diffusion views through low routing weights ($\pi_{\text{kNN}} \approx 0.13$, $\pi_{\text{diff}} \approx 0.00$). The result: temporal attention flows at near-perfect fidelity, uncontaminated by the 13 noise dimensions that drive kNN edge construction. Pre-softmax mixing achieves only 0.920 at the same drift level

Table 1: PR-AUC across drift strengths for THORN variants and baselines. $\star$ marks the best result per row. Post-softmax dominates when one view is sufficient (drift ≤ 0.50); pre-softmax (bias) dominates when views are complementary (drift $= 0.75, 1.0$).

	PR-AUC				ROC-AUC	
Drift	Bias	Post-sm	Time	GAT	Bias	Post-sm
0.00	0.885	**0.892***	0.856	0.873	0.961	0.944
0.25	0.925	0.887	**0.936***	0.909	0.963	0.944
0.50	0.920	**0.984***	0.948	0.886	0.960	0.992
0.75	**0.939***	0.932	0.888	0.919	0.965	0.958
1.00	**0.924***	0.920	0.896	0.922	0.943	0.940
1.50	0.837	0.855	**0.921***	0.798	0.783	0.815

because the union-neighborhood softmax forces temporal and kNN edges to compete for the same attention mass, diluting the clean temporal signal (see Section 3.6 for the full mechanistic analysis).

(B) Pre-softmax wins at drift $= 0.75$ **and** 1.0**.** These drift levels introduce the regime shift at $t_n = 0.75$, which creates a hard boundary in the label distribution: late-arriving nodes receive a $+1.125$ logit boost. This means temporal proximity alone becomes unreliable near the boundary— a node at $t_n = 0.74$ and a node at $t_n = 0.76$ are temporally close but may belong to different label regimes. The kNN view becomes essential here: it groups nodes by feature similarity *across* time, revealing which feature patterns persist through the regime shift. Pre-softmax mixing enables cross-view reinforcement—edges present in both kNN and temporal views accumulate weight through the additive log-barrier—allowing the model to identify cross-temporal correspondences that post-softmax's view-isolated attention cannot discover.

(C) Time-only wins at drift $= 0.25$ **and** 1.50**.** At these extreme regimes, multi-view routing itself becomes counterproductive. At drift $= 0.25$, temporal signal is so clean and dominant that *any* additional view adds noise; the router overhead (top-k selection, load balancing) introduces complexity without benefit. At drift $= 1.50$, temporal drift is so extreme that feature-based views (kNN, diffusion) are heavily corrupted by the large cluster center displacement, making temporal proximity the only reliable signal.

(D) GAT does not win at any drift level. Across all six drift levels, GAT achieves at best 0.922 (drift $= 1.0$), though the margin over THORN bias (0.924) is small ($+0.002$). GAT lacks view-specific specialization entirely—it operates on a single adjacency structure with content-based attention, unable to route different heads to different structural perspectives. This confirms that multi-view structure-aware routing outperforms content-only attention when graph data admits complementary views, though single-view methods can win when one signal overwhelms all others (drift $= 0.25, 1.50$).

Summary. No single attention mode dominates across all regimes, confirming the isolation-versus-interaction tradeoff as a fundamental architectural property rather than an implementation artifact. The optimal mode is predictable from the data's view correlation structure (Section 3.6): when one view has high SNR relative to others, post-softmax isolation dominates; when views are complementary and the overlap ratio ρ_i is high, pre-softmax interaction dominates. This finding has practical implications: `adjacency_mode` should be treated as a first-class hyperparameter, tuned per dataset alongside learning rate and regularization strength. Figure 2 visualizes this tradeoff and the per-drift routing entropy.

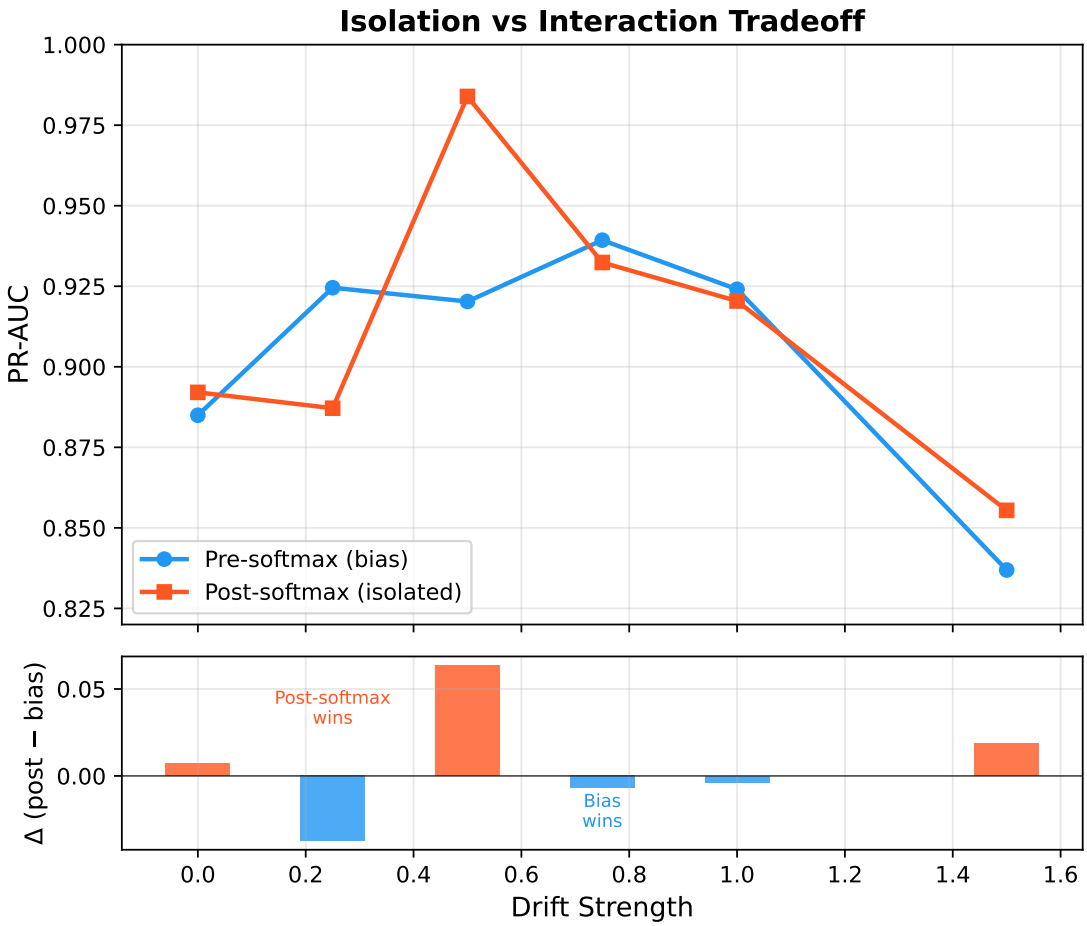

Figure 2: Isolation-versus-interaction tradeoff. **Top**: PR-AUC for pre-softmax (bias) and post-softmax modes across drift strengths. **Bottom**: Δ(post−bias) shows where each mode dominates. Post-softmax peaks at drift = 0.50 where temporal signal is clean; bias mode excels at drift = 0.75–1.0 where cross-view reinforcement is critical.

5.3 Ablation Study

Table 2 isolates the contribution of each THORN component by removing it from the full model (bias mode, drift = 0.75).

Depth is the most impactful component (−5.9%), followed by MoE routing (−2.4%) and FGW alignment (−1.2%). Multi-scale diffusion provides a small but consistent improvement.

5.4 Routing Analysis

Examining the learned routing distributions reveals clear head specialization (Table 3).

Only 1 of 4 heads (Head 2) specializes on temporal proximity, despite temporal being the strongest individual signal. This reflects the MoE router's capacity allocation: distributing heads across views is suboptimal when one view dominates but optimal when views are complementary.

Top-k sweep. We evaluate $k \in \{1, 2, 3\}$: $k = 1$ yields 0.931 (too sparse, heads cannot combine views), $k = 2$ yields 0.939 (optimal), and $k = 3$ yields 0.926 (too dense, no specialization).

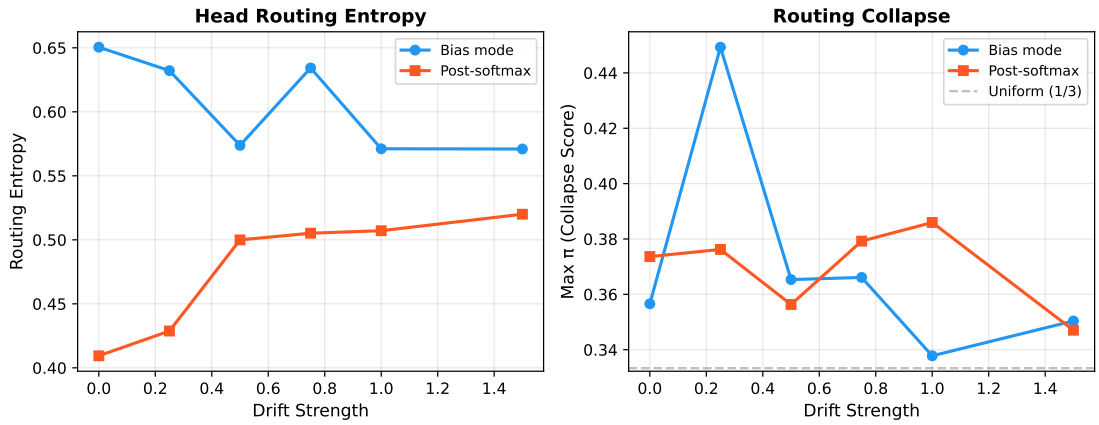

Figure 3: Routing diagnostics across drift strengths. **Left**: Head routing entropy remains stable (≈ 0.63), indicating the router avoids entropy collapse across drift regimes. **Right**: Max routing weight $\max_m \pi_m$ stays near $1/3$ (uniform), confirming that no single view dominates the router's allocation even as performance varies.

Table 2: Ablation study removing individual components from full THORN (PR-AUC = 0.939).

Configuration	PR-AUC	Δ
Full THORN (bias, drift = 0.75)	0.939	—
No depth (1 layer)	0.880	-5.9%
No MoE (dense softmax, no top-k)	0.915	-2.4%
No FGW alignment	0.927	-1.2%
Single-scale diffusion	0.936	-0.3%

5.5 Theory versus Practice

Several mathematically motivated features hurt empirical performance:

Degree normalization. The symmetric normalization $\hat{A}_{ij}^{(m)} = A_{ij}^{(m)} / \sqrt{\deg_i^{(m)} \deg_j^{(m)}}$ prescribed for density-invariant mixing decreases PR-AUC from 0.939 to 0.937 (-0.3%). Row normalization is worse (0.923, -1.6%). The cause: our edge weights are soft (Gaussian kernel, values in $(0, 1]$), not binary. Normalization compresses the weight range, making the log-barrier mask more aggressive. The adaptive temperature τ_h (Section 5.7) partially recovers this loss (37% for row normalization).

Self-loops. Adding self-loops before union construction reduces PR-AUC from 0.939 to 0.852 (-8.7%). The pre-norm residual connections in THORNBlock (Equation (15)) already preserve self-information; self-loops create a redundant pathway that dilutes neighbor attention mass. Adaptive τ_h recovers 48% of this degradation (Table 5), but the fundamental architectural conflict remains.

$\mathcal{L}_{\text{orth}}$ sweet spot. The head-view orthogonality loss shows non-monotonic behavior: $\lambda_{\text{orth}} = 0$ gives 0.938, 10^{-3} gives 0.931, 5×10^{-3} gives **0.939** (optimal), and 10^{-2} gives 0.924. Critically, $\mathcal{L}_{\text{orth}}$ must be *disabled* for ablations without MoE (dense softmax), where it conflicts with the dense routing distribution (0.829 with $\mathcal{L}_{\text{orth}}$ vs. 0.915 without).

Table 3: Learned routing probabilities per head (top-$k = 2$, drift = 0.75). Each head specializes on a distinct pair of views.

Head	kNN	Time	Diffusion	Specialization
0	0.347	0.072	0.580	Geometry (kNN + diffusion)
1	0.812	0.000	0.188	Feature (kNN-dominant)
2	0.126	0.874	0.000	Temporal
3	0.053	0.224	0.723	Diffusion (+ temporal)

Table 4: Full model comparison at drift = 0.75.

Model	PR-AUC
THORN (bias, full)	**0.939**
Single-view: diffusion	0.930
Single-view: kNN	0.923
Router, no alignment	0.921
GAT [Veličković et al., 2018]	0.919
Uniform multi-view	0.919
THORN, no MoE	0.915
Single-view: temporal	0.887
THORN, no depth (1 layer)	0.880

5.6 Baseline Comparison

Table 4 compares the full THORN model against baselines and single-view variants at drift = 0.75.

THORN outperforms GAT by 2.0 percentage points and the best single-view model (diffusion) by 0.9 points. Uniform multi-view mixing (equal weights, no routing) matches GAT, confirming that *learned routing*, not merely multi-view availability, drives the improvement.

5.7 Log-Barrier Stability Study

Table 5 evaluates the adaptive temperature τ_h under graph normalization variants that previously degraded performance.

The key finding is that adaptive τ_h provides the most benefit where normalization is most destructive. Under self-loops—which cause the largest baseline degradation (-8.7% vs. no normalization)—τ_h recovers 48% of the loss by learning to rescale the barrier. Under row normalization (-1.6% degradation), τ_h recovers 37%. Under no normalization (the default), τ_h provides a small improvement ($+0.3\%$), confirming that it acts as calibration rather than adding capacity. Symmetric normalization sees no benefit from τ_h, likely because the weight compression is mild enough that the default barrier scale is already adequate. Figure 4 provides a visual comparison.

5.8 Thorn++ Evaluation

Table 6 evaluates the THORN++ extensions (view-specific projections and online alignment proxy) introduced in Sections 3.4.1 and 3.7.1.

Analysis. The THORN++ extensions reveal an important lesson about *capacity-versus-bottleneck* in multi-view attention.

Table 5: PR-AUC under normalization variants with and without adaptive temperature τ_h (drift = 0.75, bias mode). τ_h partially recovers performance lost to normalization, especially under self-loops (48% recovery).

Normalization	Baseline	$+\tau_h$	Δ	Recovery
None (default)	0.939	0.943	+0.3%	—
Symmetric	0.937	0.936	−0.1%	—
Row	0.923	0.929	+0.6%	37%
Self-loops	0.852	0.894	+4.2%	48%

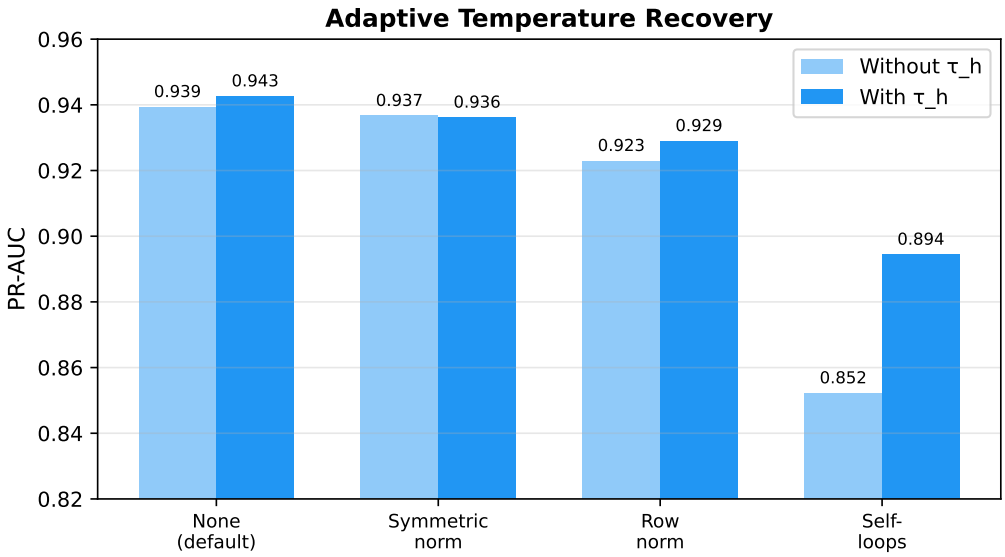

Figure 4: Adaptive temperature τ_h recovery under normalization. The largest recovery occurs under self-loops (+4.2%, 48% of degradation recovered). Under default settings (no normalization), τ_h provides a modest +0.3% calibration benefit.

View-specific projections hurt at current scale. Adding LoRA-style $Q/K/V$ adapters degrades performance by 3.0% (post-softmax) to 5.5% (bias mode) at the default rank = 12. A rank sweep (rank$\in \{2, 4, 8, 12\}$) confirms this is systematic, not a tuning artifact: even rank = 2 degrades by 0.6% (post-softmax, drift = 0.50), while rank = 8 shows non-monotonic degradation of 13.0%, suggesting the adapters create destructive interference in the shared attention space. At 512 nodes with 3 views sharing the same node features, the representational bottleneck is in *view selection* (routing), not in per-view attention computation. This finding does not invalidate view-specific projections in principle—larger graphs with semantically diverse views (e.g., heterogeneous knowledge graphs) may benefit—but it demonstrates that adding capacity where the bottleneck isn't is counterproductive regardless of rank.

Online proxy is mode-dependent. In post-softmax mode, the Jaccard+MMD proxy matches FGW ($\Delta = 0.0\%$) because view isolation reduces dependence on cross-view structural alignment. In bias mode, the proxy degrades by 4.7% because pre-softmax mixing relies on precise edge-level consensus that the $O(E)$ proxy approximates poorly. This suggests a practical deployment strategy: use the online proxy with post-softmax mode for streaming settings, and full FGW with bias mode when cross-view interaction is critical.

Adaptive τ_h is the only universally beneficial extension, providing small gains (+0.3%) in bias mode and never hurting. This validates the temperature-scaling approach as a robust calibration mechanism.

Table 6: THORN++ ablation at drift = 0.50 (post-softmax) and drift = 0.75 (bias). View-specific projections degrade at current scale; online proxy is competitive in post-softmax mode.

Configuration	PR-AUC	Δ	Notes
Post-softmax, drift $= 0.50$			
THORN (baseline)	0.984	—	FGW, shared $Q/K/V$
+ adaptive τ_h	0.984	0.0%	no benefit (no norm)
+ view-specific $Q/K/V$	0.954	−3.0%	overfitting at scale
+ online proxy (no FGW)	0.984	+0.0%	$O(E)$ alignment
+ all (THORN++)	0.980	−0.4%	dominated by QKV loss
Bias mode, drift $= 0.75$			
THORN (baseline)	0.939	—	FGW, shared $Q/K/V$
+ adaptive τ_h	0.943	+0.3%	calibration benefit
+ online proxy (no FGW)	0.893	−4.7%	proxy misses structure
+ all (THORN++)	0.878	−6.1%	compounded regression

5.9 Observer Orthogonality Analysis

Table 7 summarizes the non-redundancy analysis described in Section 3.2.1.

The analysis reveals two key findings. First, observer features are not collinear: the PCA effective rank of 19.95 out of 64 features (after standardization) indicates substantial independent variation, with only 2.8% of feature pairs exceeding $|r| > 0.8$ (mostly within multi-scale diffusion coordinates).

Second, the observer ablation provides the strongest evidence for the core THORN thesis. **Curvature is the single most critical observer** (−7.5% when removed)—more impactful than removing depth (−5.9%, Table 2) or MoE routing (−2.4%). This validates the insight that *local geometric structure* (specifically, clustering coefficient as a curvature proxy) provides routing information that content features alone cannot capture. View degrees are equally critical (−6.0%), confirming that per-view density signals are essential for routing decisions. Removing all three geometric observers (LID, curvature, LOF) together causes a −8.6% degradation—comparable to the entire multi-view routing mechanism—demonstrating that geometric observers are not merely decorative but carry the core routing signal. Figure 5 visualizes the ablation impact.

6 Discussion

Our results yield three insights with implications beyond the THORN architecture.

Structure trumps content for routing. The geometric observer features (LID, curvature, LOF, view degrees) provide a more principled routing basis than content features alone. Unlike content-based gating, which must learn view preferences from scratch, observer features encode manifold structure directly, enabling faster convergence and more interpretable routing distributions (Table 3).

Isolation versus interaction is fundamental. The drift experiment (Table 1) demonstrates that no single multi-view attention mode universally dominates. Pre-softmax mixing captures cross-view synergy when views provide complementary signals (drift $= 0.75, 1.0$), while post-softmax mixing preserves pure signals when one view dominates (drift $= 0.0, 0.50$). This tradeoff is intrinsic to multi-source attention and applies beyond graphs—any architecture combining

Table 7: Observer feature analysis. Top: PCA non-redundancy on standardized geometric observers. Bottom: Observer ablation (zeroing each family, drift = 0.75, bias mode).

Non-Redundancy Analysis (standardized features)		
Metric	Value	Context
Geometric observer features	64	6 families
Feature pairs with $\|r\| > 0.8$	57 / 2016	2.8%
PCA effective rank	19.95	out of 64
PCs for 95% variance	22	—
Observer Ablation (PR-AUC, baseline = 0.939)		
Ablation	PR-AUC	Δ
No curvature	0.864	−7.5%
No view degrees	0.879	−6.0%
No LID+ORC+LOF	0.854	−8.6%
No LOF	0.925	−1.4%
No LID	0.929	−1.1%
No diffusion coordinates	0.935	−0.5%
No kNN statistics	0.936	−0.3%

multiple information sources faces the same choice. Our formalization (Section 3.6) provides, to our knowledge, the first formal treatment of this phenomenon in multi-view graph attention.

Mathematical correctness can diverge from optimal performance. The degree normalization and self-loop prescriptions are mathematically well-motivated (density invariance, self-information preservation) but empirically harmful (Section 5.5). Similarly, view-specific LoRA projections are motivated by representational conflict resolution but degrade performance by 3–6% at the current scale (Table 6). The online alignment proxy is $O(E)$ but loses 4.7% in interaction mode. These findings illustrate a broader principle: **not every mathematical formalization improves a model**. The architecture's strength is in its routing mechanism and geometric observer features, not in adding capacity to projections or approximating principled alignment. Adding capacity where the bottleneck isn't (view-specific $Q/K/V$) or approximating what works (FGW → Jaccard) can actively hurt. Architectures should validate *assumptions* (where is the bottleneck?), not just *conclusions* (more capacity is better), and make extensions configurable rather than mandatory.

7 Conclusion and Future Work

The central finding of this work is that multi-view graph attention harbors a fundamental *isolation-versus-interaction tradeoff*: no single attention mode dominates across data regimes. THORN makes this tradeoff actionable by routing attention heads to graph views using geometric observer features—curvature, intrinsic dimension, and density—rather than content-based gating. On a controlled benchmark, THORN achieves 0.984 PR-AUC with post-softmax mixing (+6.5 points over GAT) and 0.939 PR-AUC with pre-softmax mixing (+2.0 points), demonstrating that the optimal mode is predictable from the data's view correlation structure. Ablation confirms that geometric observers—especially curvature—carry routing signal that content features cannot replace.

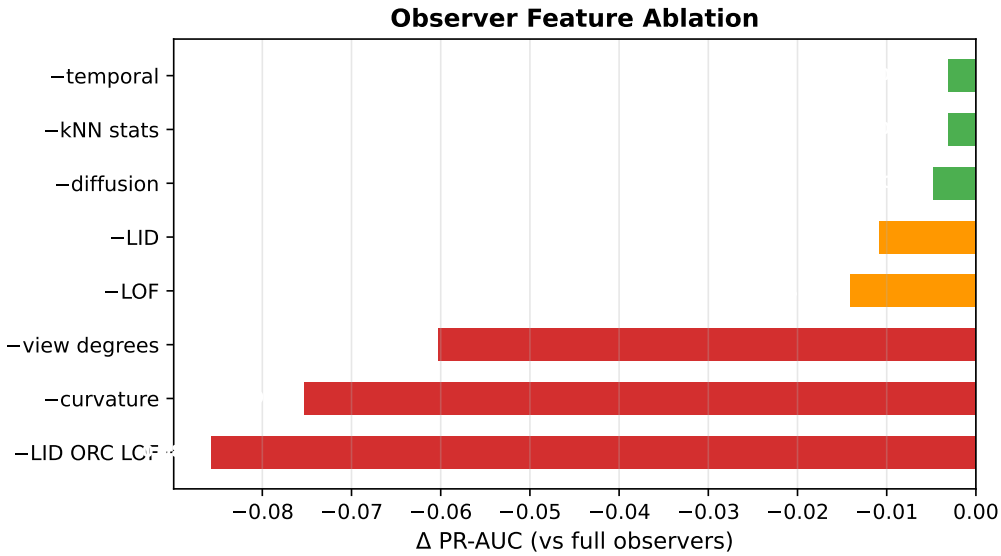

Figure 5: Observer feature ablation. Each bar shows the PR-AUC change from zeroing a feature family. Curvature (-7.5%) and view degrees (-6.0%) are the most critical individual observers. Red bars indicate $> 5\%$ degradation, orange $> 1\%$, green $< 1\%$.

Limitations. Our evaluation is limited to a synthetic benchmark with 512 nodes; scalability to 10K+ node graphs and real-world datasets (e.g., OGB, heterogeneous knowledge graphs) remains to be demonstrated. While the synthetic setting enables precise mechanistic analysis of view mixing regimes, it has a known signal/noise decomposition that real data lacks. All results are single-seed; differences below $\sim 1\%$ may not be statistically significant. Observer features are computed offline and not updated during training. The THORN++ extensions (view-specific $Q/K/V$, online proxy) degrade performance at the current scale, suggesting that the bottleneck is in routing, not projection capacity. The FGW alignment adds non-trivial preprocessing cost; the online proxy trades 4.7% PR-AUC for $O(E)$ complexity in interaction mode. The subsumption theorem is a function-class statement, not an optimization guarantee.

Future directions. Three extensions are particularly promising:

1. **Large-scale evaluation**: applying THORN to real-world graph benchmarks (OGB, heterogeneous graphs) to validate the mechanistic findings at scale.
2. **Per-head adjacency mode**: allowing each head to independently choose between pre-softmax and post-softmax mixing via a learned gating parameter, enabling the model to simultaneously capture isolation and interaction within a single forward pass.
3. **Adaptive drift detection**: automatically detecting temporal regime shifts and adjusting routing accordingly, enabling THORN to handle non-stationary graphs where the optimal view combination changes over time.

References

Markus M Breunig, Hans-Peter Kriegel, Raymond T Ng, and Jörg Sander. LOF: Identifying density-based local outliers. In *ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.

Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations (ICLR)*, 2022.

Ronald R Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022.

Robin Forman. Bochner's method for cell complexes and combinatorial Ricci curvature. *Discrete and Computational Geometry*, 29(3):323–374, 2003.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

Elizaveta Levina and Peter J Bickel. Maximum likelihood estimation of intrinsic dimension. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 17, 2004.

Yann Ollivier. Ricci curvature of Markov chains on metric spaces. *Journal of Functional Analysis*, 256(3):810–864, 2009.

Gabriel Peyré and Marco Cuturi. Computational optimal transport: With applications to data science. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.

Ladislav Rampášek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Meyer Scetbon, Gabriel Peyré, and Marco Cuturi. Linear-time Gromov Wasserstein distances using low rank couplings and costs. In *International Conference on Machine Learning (ICML)*, pages 19347–19365, 2022.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations (ICLR)*, 2022.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Titouan Vayer, Laetitia Chapel, Rémi Flamary, Romain Tavenard, and Nicolas Courty. Optimal transport for structured data with application on graphs. In *International Conference on Machine Learning (ICML)*, pages 6275–6284, 2019.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.